

1. The Standard Syntax and The "Always Brace" Rule

At its core, an if statement evaluates a boolean condition and executes a block of code if that condition is true.

Java

```
int confidenceScore = 85;

if (confidenceScore >= 90) {
    System.out.println("High accuracy OCR text extraction.");
} else if (confidenceScore >= 70) {
    System.out.println("Requires manual review.");
} else {
    System.out.println("Scan failed. Retake image.");
}
```

The Professional Standard: Always use curly braces {}, even if the execution block is only a single line.

While Java allows you to omit them for single lines, doing so is considered a critical security vulnerability (famously causing the Apple "goto fail" bug). If another developer adds a second line later without adding the braces, that second line will execute unconditionally.

2. The Boolean Anti-Pattern

Never compare a boolean variable to true or false using ==. The variable itself is already a boolean expression.

Java

```
boolean isImageClear = true;

// ❌ Amateur / Bad Practice
if (isImageClear == true) {
    // ...
}

// ❌ Amateur / Bad Practice
if (isImageClear == false) {
    // ...
}

// ✅ Professional Standard
if (isImageClear) {
```

```

    // ...
}

// ☐ Professional Standard (Use the ! NOT operator)
if (!isImageClear) {
    // ...
}

```

3. Guard Clauses (The Early Return)

Deeply nested if statements create what is known as the "Arrow Anti-Pattern" (code that points heavily to the right). It is incredibly difficult to read. Professional enterprise code, especially in microservices or API development, relies heavily on **Guard Clauses**.

A Guard Clause immediately checks for invalid conditions and exits the method (return), keeping the "happy path" un-nested at the bottom.

❌ Bad Practice (Deep Nesting):

Java

```

public void processScannedDocument(Document doc) {
    if (doc != null) {
        if (doc.isReadable()) {
            if (doc.hasText()) {
                System.out.println("Extracting text for ClearFlow AI...");
            } else {
                System.out.println("Document is blank.");
            }
        } else {
            System.out.println("Image is too blurry.");
        }
    } else {
        System.out.println("No document provided.");
    }
}

```

✅ Professional Standard (Guard Clauses):

Java

```

public void processScannedDocument(Document doc) {
    // 1. Guard against null or invalid states immediately
    if (doc == null) {

```

```

        System.out.println("No document provided.");
        return;
    }
    if (!doc.isReadable()) {
        System.out.println("Image is too blurry.");
        return;
    }
    if (!doc.hasText()) {
        System.out.println("Document is blank.");
        return;
    }

    // 2. The "Happy Path" is clean and un-nested
    System.out.println("Extracting text for ClearFlow AI...");
}

```

Interview Vault: Tricky and Viral Questions

These questions are designed to catch developers who rely too heavily on IDE autocomplete without understanding the underlying syntax rules.

Q: "What is wrong with the following code, and what will it print?"

Java

```

boolean isAdmin = false;
if (isAdmin = true) {
    System.out.println("Access Granted");
} else {
    System.out.println("Access Denied");
}

```

Answer: It will print "**Access Granted**".

The Trap: The developer used a single equals sign (=) instead of the equality operator (==). In Java, `isAdmin = true` is an *assignment*. It assigns true to `isAdmin`, and then the if statement evaluates the result of that assignment (which is true). It compiles perfectly but creates a massive logic and security flaw.

Q: "What will the output be? Explain the 'Dangling Else' problem."

Java

```
boolean isActive = true;
boolean hasLicense = false;

if (isActive)
    if (hasLicense)
        System.out.println("System Running");
else
    System.out.println("System Stopped");
```

Answer: It prints "**System Stopped**".

The Trap: The indentation makes it look like the else belongs to the first if (isActive). However, the Java compiler ignores whitespace. The hard rule is: **an else always binds to the nearest preceding if that does not have an else**. Without curly braces to explicitly define the blocks, the else binds to if (hasLicense).

Q: "What is the difference between & and && in an if condition?"

Java

```
// Scenario A
if (user != null && user.getRole().equals("Admin")) { ... }

// Scenario B
if (user != null & user.getRole().equals("Admin")) { ... }
```

Answer: && is the **Logical Short-Circuit AND**. & is the **Bitwise AND**.

The Trap: If user is null in Scenario A, the && operator evaluates the left side (user != null), sees that it is false, and immediately *short-circuits* (stops evaluating).

In Scenario B, the & operator forces the evaluation of *both* sides regardless of the first result. If user is null, evaluating the right side (user.getRole()) will throw a fatal NullPointerException and crash the application. Always use && for control flow logic.